

JUDICIARY INFORMATION SYSTEM

Software Engineering Project Evaluation Deck

A role-based digital records platform for case registration, hearing management, judicial archives and auditability.

NEXT.JS 15

TYPESCRIPT

PRISMA

SUPABASE

Course: Software Engineering

Evaluation focus: requirements, design, architecture, data, security and testing



JIS

Project snapshot

21

Functional requirements

10

Non-functional requirements

48

Unit tests - reported 100% pass

11

End-to-end integration tests

WHO USES IT

Registrar

Creates accounts, manages cases, schedules hearings, records judgments, audits and analytics.

Judge

Researches closed case archives at no charge.

Lawyer

Researches closed cases with ₹50 per inspection and billing ledger.

Scope intentionally prioritizes the artefacts an evaluator can inspect: requirements, architecture, UML/DFD, schema, access control, validation and tests.

Problem statement

CURRENT PAIN POINTS

- Lost or misplaced case files across manual and fragmented records.
- Scheduling conflicts: judges can be double-booked and courtrooms can be overbooked.
- No centralized searchable archive for precedent research.
- Limited transparency around billable lawyer access to closed cases.
- Administrative actions lack a single immutable audit trail.

SYSTEM RESPONSE

Centralize the case lifecycle and enforce the rules in code.

- 01 Digital case register
- 02 Slot + judge conflict checks
- 03 Read-only closed-case archive
- 04 Per-view billing log
- 05 Append-only audit history

Objectives and success criteria

01

Secure RBAC

Registrar / Judge / Lawyer permissions are enforced.

02

CIN generation

Every new case gets an automatically generated identifier.

03

Schedule integrity

Courtroom capacity and judge conflicts are checked before booking.

04

Immutable history

Case mutations produce timestamped history and audit records.

05

Archive research

Closed cases are searchable and read-only.

06

Billable access

Lawyer inspections create a ₹50 CaseView + audit entry.

07

Usable records UI

Government-records aesthetic with clear operational workflows.

08

Maintainable stack

Typed schema, layered architecture, reusable server actions and validators.

Evaluation point: the project translates each objective into a concrete business rule, action, schema relation or test case.

Requirements coverage

FUNCTIONAL MODULES

A	User & Account Management	Create/deactivate/reactivate users; role control; lawyer view log.
B	Case Registration	Case data capture + automatic CIN generation + personnel assignment.
C	Hearing Scheduling	Available slots, capacity checks, judge conflict detection, adjournments.
D	Case Resolution	Judgment entry and permanent read-only closure.
E	Registrar Queries	Pending, resolved, hearing-date and CIN-based queries.
F	History & Search	Closed-case archive and keyword search.
G	Audit & History	Immutable, timestamped hearing/judgment/adjournment history.

21 FR

10 NFR

Security • integrity • availability • performance • usability • scalability • auditability • maintainability • accessibility • portability

Core case lifecycle



Adjournment path: SCHEDULED -> ADJOURNED -> optional new hearing -> PENDING, while preserving the original history.

CaseStatus: REGISTERED → PENDING ↔ ADJOURNED → CLOSED

Actors and access model

REGISTRAR

- Full case lifecycle control
- User + courtroom management
- Audit trail + analytics
- Search across all statuses

JUDGE

- Closed-case research
- Free case inspection
- Keyword search on archive
- No mutation privileges

LAWYER

- Closed-case research
- ₹50 per inspection
- Every view logged
- Billing ledger available

THREE ENFORCEMENT LAYERS

1 **Middleware**

Unauthenticated -> /login;
wrong role -> own dashboard

2 **Server Actions**

getCurrentUser() + explicit
permission checks

3 **Database**

Row-Level Security policies as
defense-in-depth

Technology stack

FRONTEND / FULL STACK

Next.js 15 App Router

Server Components + Server Actions

UI

React 19 + Tailwind CSS 4 + shadcn/ui

Accessible reusable components

AUTH

Supabase Auth

Email/password + sessions

TESTING

Vitest + Playwright

Unit + browser E2E

LANGUAGE

TypeScript 5.6+

Typed UI and backend

DATA

Supabase Postgres + Prisma 5.x

Managed PostgreSQL + type-safe ORM

VALIDATION

react-hook-form + Zod

Client + server schema validation

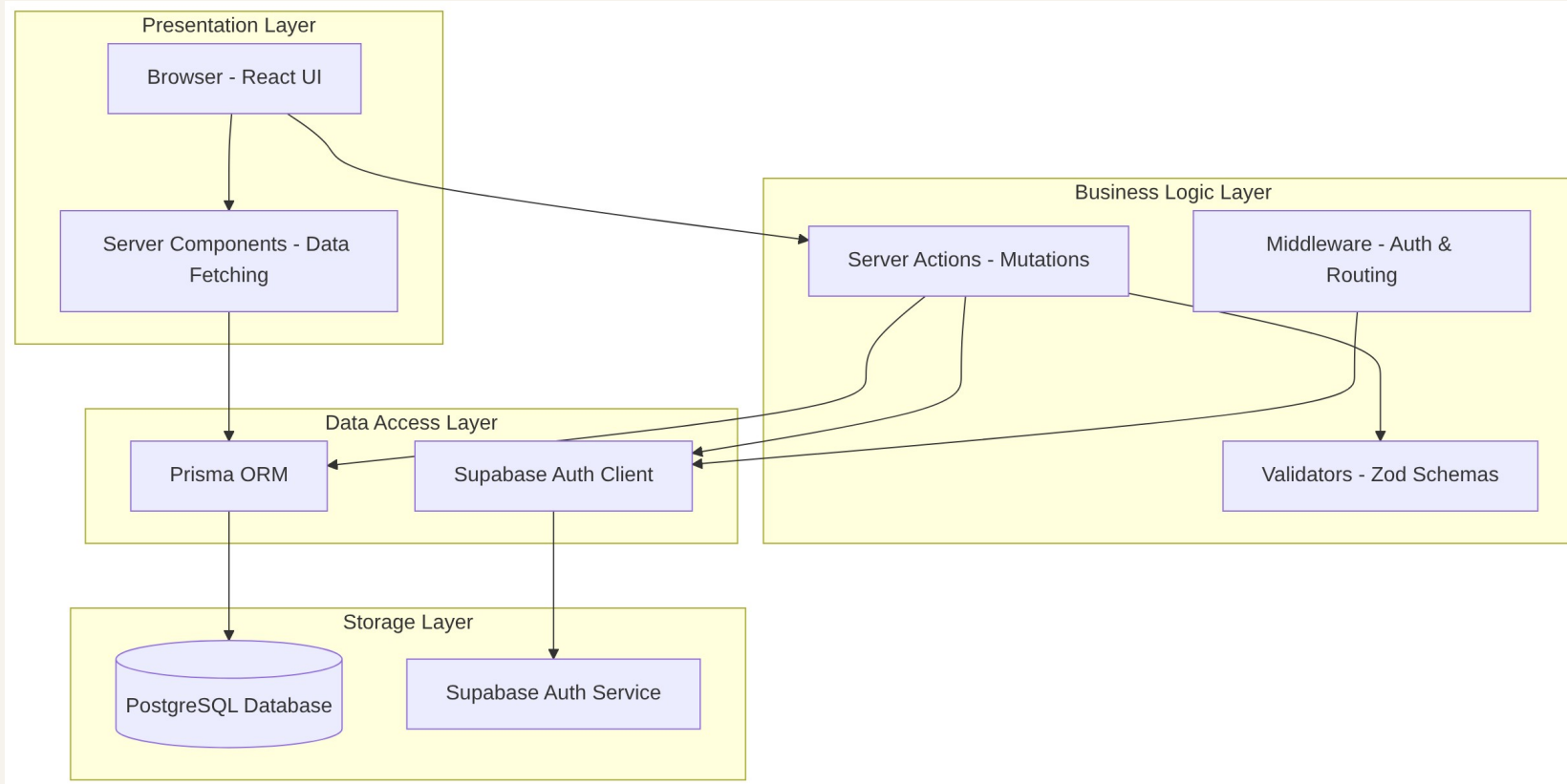
DEPLOYMENT

Vercel + Supabase Cloud

Serverless hosting + managed DB

Design intent: avoid a separate API server. Typed Server Actions connect the UI layer to validation and Prisma-backed persistence.

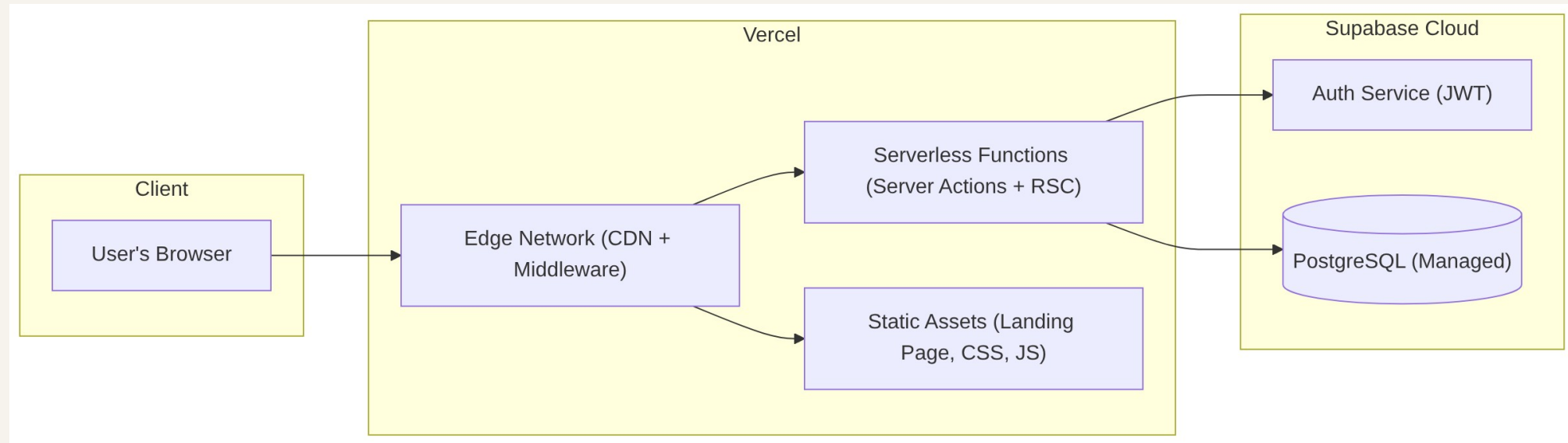
Layered / 3-tier architecture



WHY THIS MATTERS

- UI fetches data with Server Components.
- Mutations pass through Server Actions.
- Zod validates at the boundary.
- Prisma centralizes database access.
- Supabase Auth handles sessions.
- Postgres is the durable system of record.

Production deployment architecture

**Client**

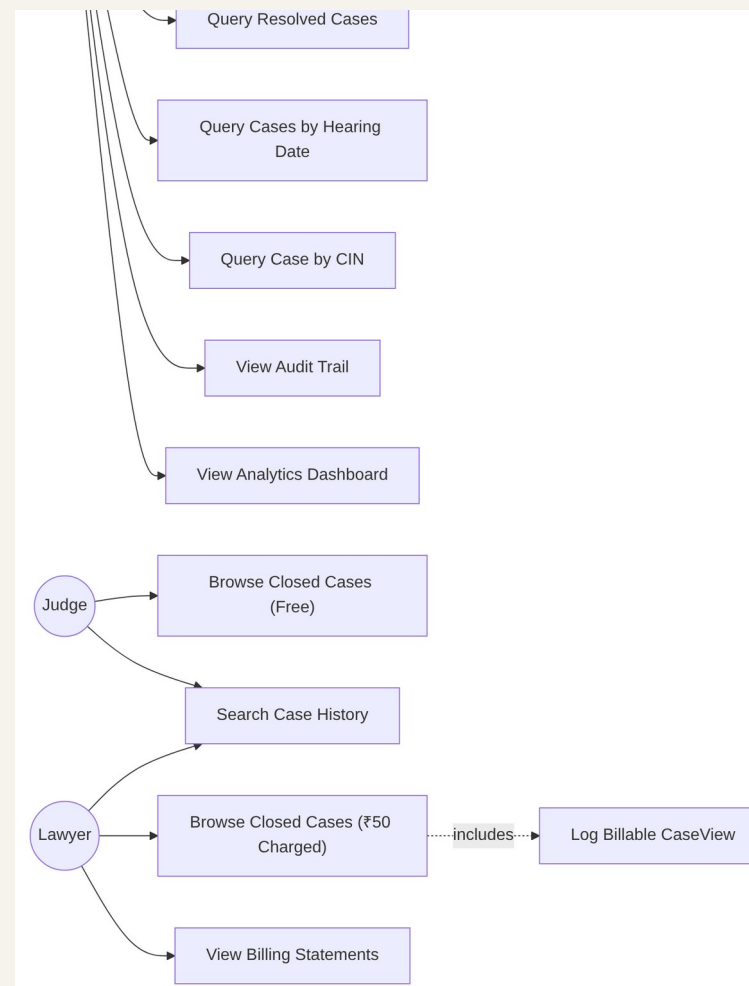
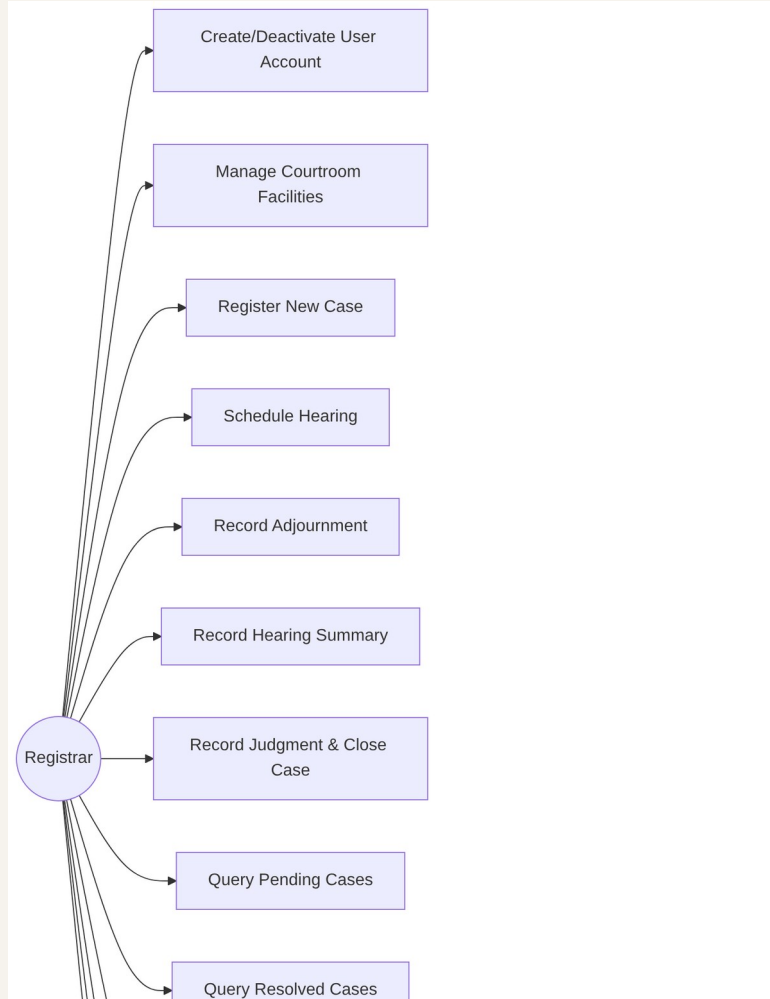
User browser

VercelCDN/Edge + middleware + serverless
RSC/actions**Supabase**

Auth service + managed PostgreSQL

Production configuration is environment-driven: DATABASE_URL, DIRECT_URL, Supabase URL/anon key and service role key.

Use case model



EVALUATOR READOUT

Registrar

Operational control: accounts, facilities, case lifecycle, audit and analytics.

Judge

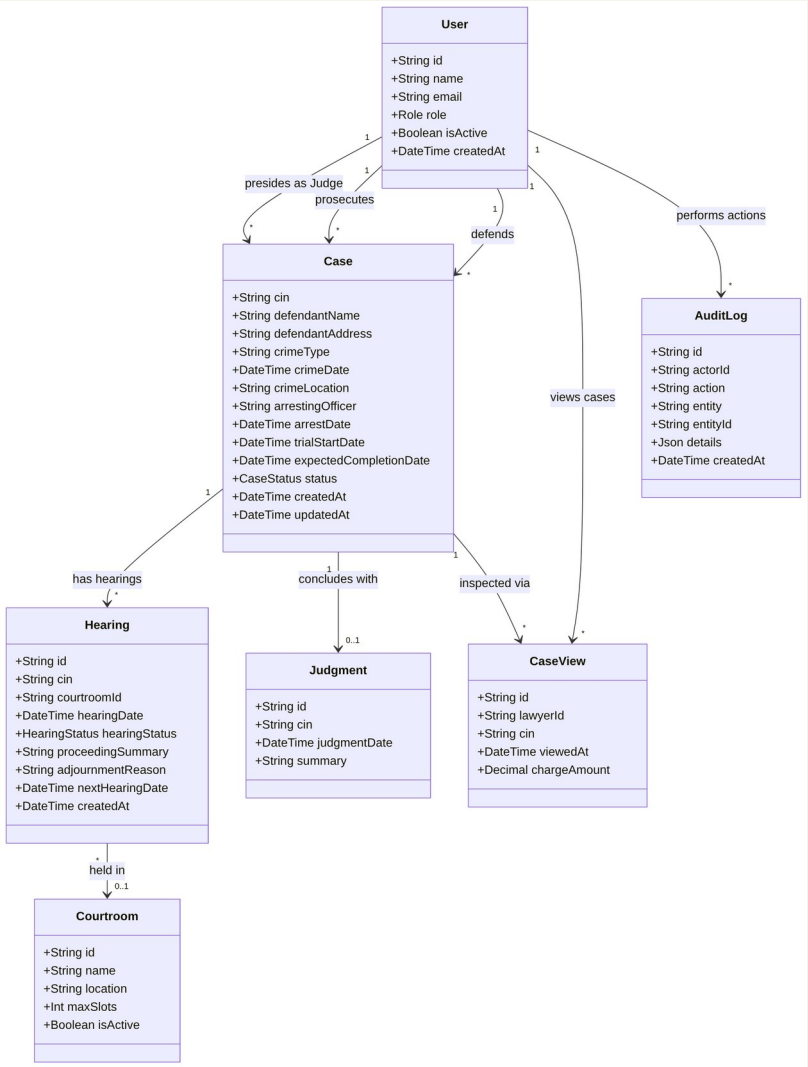
Research-only access to closed cases, without mutation or billing.

Lawyer

Archive access is billable per inspection and creates a CaseView record.

The use-case model makes role boundaries visible before the evaluator inspects the implementation.

Domain model



CORE ENTITIES

User	roles, activity, identity
Case	defendant + crime + personnel + status
Hearing	date, courtroom, summary, adjournment
Judgment	1:1 resolution record
CaseView	lawyer billing event
AuditLog	actor + action + entity + details

Key relationship: Case 1→* Hearing, Case 1→0..1 Judgment, Case 1→* CaseView.

Database design

User id, name, email, role, isActive

Courtroom id, name, location, maxSlots, isActive

Case cin, defendant, crime, dates, status, personnel FKs

Hearing cin, courtroomId, hearingDate, status, summary

Judgment cin, judgmentDate, summary

CaseView lawyerId, cin, viewedAt, chargeAmount

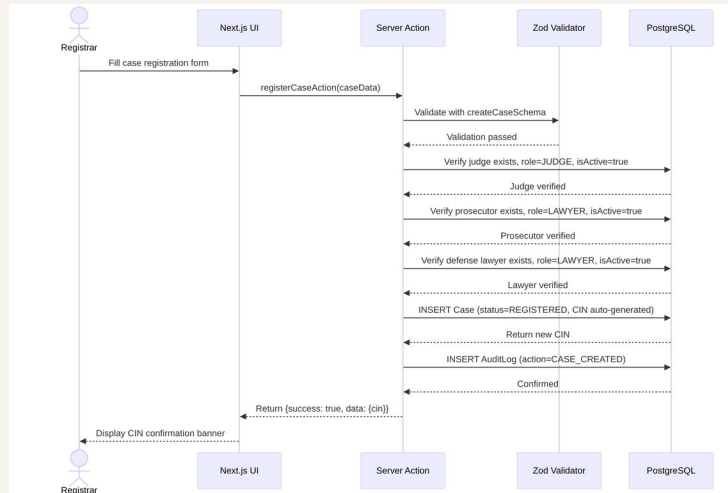
AuditLog actorId, action, entity, entityId, details, createdAt

INDEXES CALLED OUT

Case: status / judge / lawyer / crimeType / defendantName / trialStartDate
Hearing: cin / hearingDate / (courtroomId, hearingDate)
Audit: actorId / (entity, entityId) / createdAt

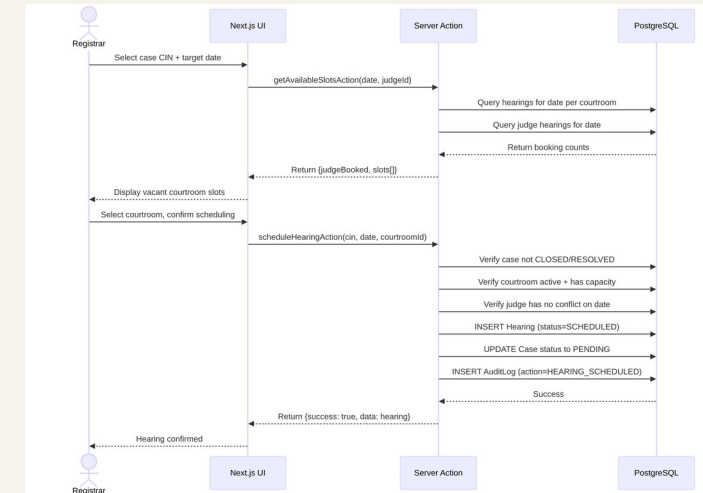
The schema supports append-only case history while keeping transactional state in Case and event/history records in Hearing, Judgment, CaseView and AuditLog.

Critical interaction flows



REGISTER CASE

Validate -> verify personnel -> INSERT Case -> generate CIN -> AuditLog

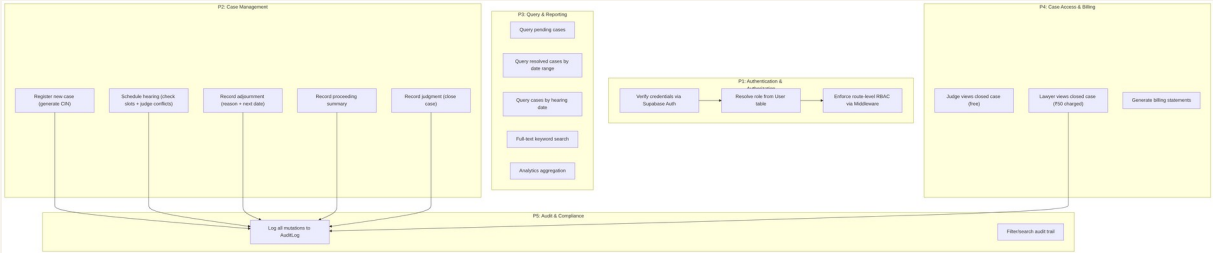
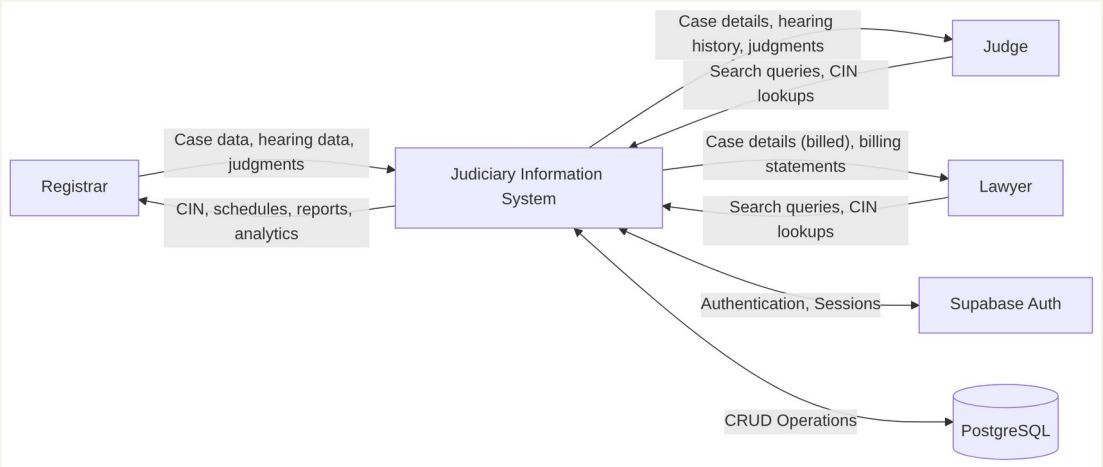


SCHEDULE HEARING

Compute vacant slots -> check judge conflict -> check capacity -> INSERT Hearing -> AuditLog

The same pattern continues into lawyer billing, adjournment and judgment recording: validate, enforce business rules, persist, audit.

Data flow views



LEVEL 0

External actors exchange case data, searches and results with JIS; Auth and PostgreSQL are explicit supporting systems.

LEVEL 1

Processes are grouped into authentication, case management, query/reporting, case access/billing and audit/compliance.

API and server-action surface

Authentication

loginAction, logoutAction

User management

createUserAction, deactivateUserAction, reactivateUserAction, getAllUsersAction

Courtrooms

createCourtroomAction, updateCourtroomAction, toggleCourtroomActiveAction, getAllCourtroomsAction

Case management

registerCaseAction, recordJudgmentAction, queries, getCaseByCinAction, searchCasesAction

Hearings

getAvailableSlotsAction, scheduleHearingAction, recordAdjournmentAction, recordHearingSummaryAction

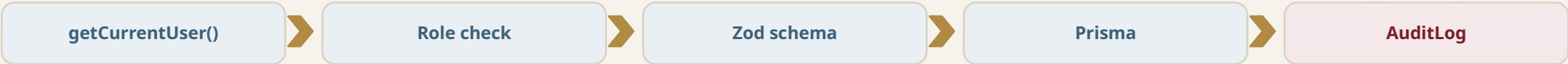
Case access

viewCaseAsLawyerAction, viewCaseAsJudgeAction, getLawyerBillingAction, getClosedCasesAction

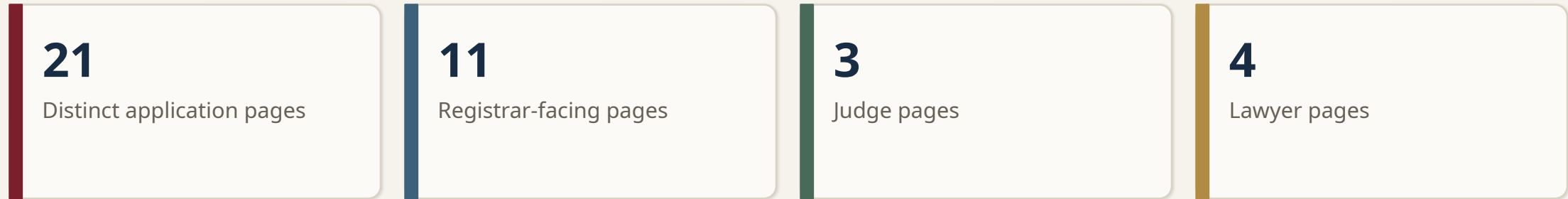
Audit / analytics

getAuditLogsAction, getAuditFilterOptionsAction, getRegistrarAnalyticsAction

Standard action contract: authenticate -> authorize -> Zod validate -> Prisma operation -> AuditLog for writes -> return success/error payload.



Application pages and UI

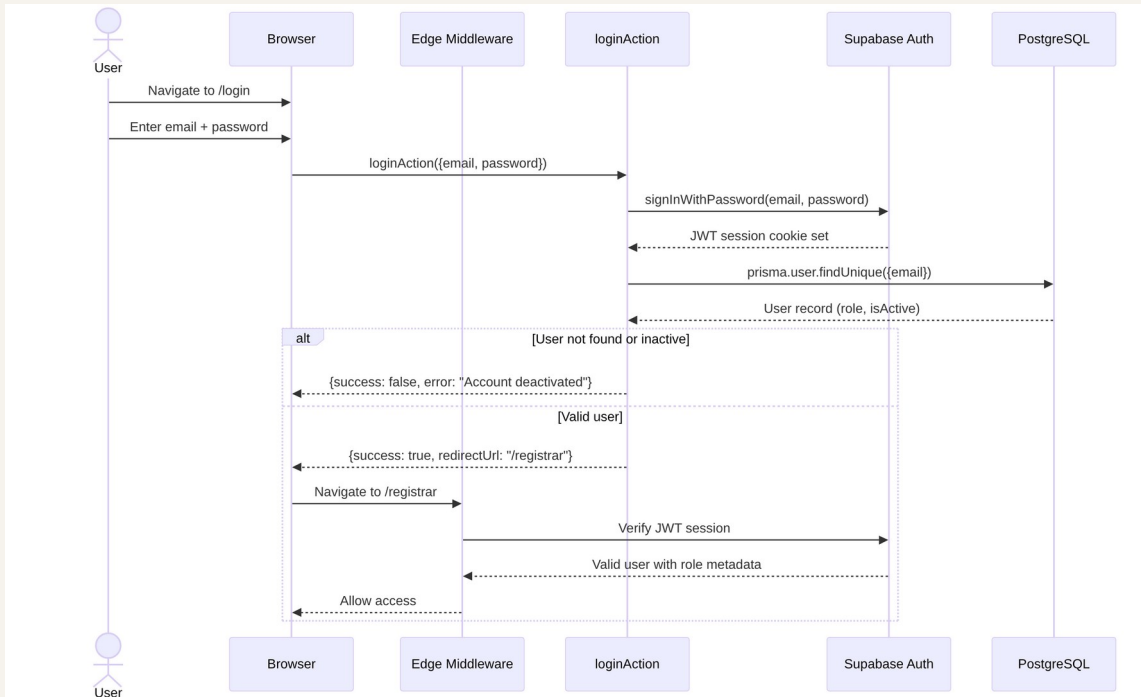


REPRESENTATIVE EVALUATOR CHECKPOINTS

/registrar	Dashboard	KPIs, today's hearings, audit feed, analytics preview
/registrar/cases/new	Case registration	Multi-section form, personnel dropdowns, date rules
/registrar/cases/[cin]	Case detail	Docket + hearing timeline + scheduling/adjournment/judgment
/registrar/audit	Audit trail	Paginated filters + JSON detail expansion
/judge/history/[cin]	Judge inspection	Full closed-case detail, read-only
/lawyer/history/[cin] + /billing	Lawyer archive + billing	Billed inspection + itemized transaction ledger

The visual layer follows a professional government-records aesthetic rather than a generic SaaS dashboard.

Security, authentication and validation



AUTHENTICATION

Supabase Auth session + Prisma User lookup resolves role and isActive state.

VALIDATION

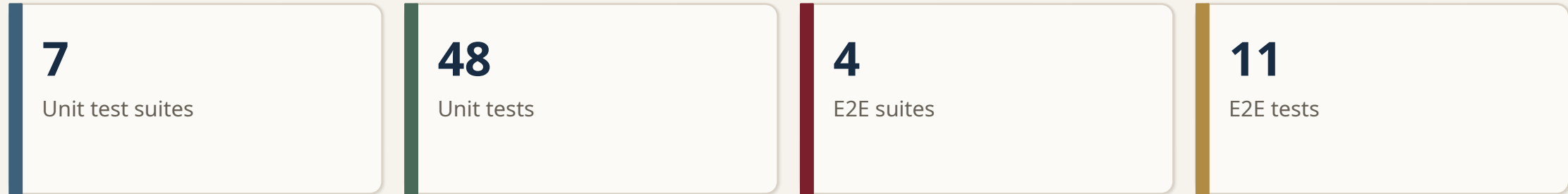
Zod schemas are reused on client and server; date relationships are checked with cross-field rules.

INTEGRITY

Writes log to AuditLog; judgment creation + case closure run in a transaction.

Edge cases explicitly tested/handled include inactive users, wrong-role assignments, full courtrooms, judge conflicts, duplicate judgments and lawyer access to active cases.

Testing and edge-case coverage



REPRESENTATIVE BUSINESS-RULE TESTS

- Reject non-Registrar user administration and case registration.
- Prevent duplicate judgments on closed cases.
- Reject invalid personnel roles and inactive assignees.
- Bill ₹50 and create a CaseView on every lawyer inspection.
- Detect judge double-booking and full courtroom capacity.
- Handle zero-data analytics without division-by-zero.
- Require a hearing before recording a judgment.
- Verify unauthenticated dashboard routes redirect to /login.

REPORTED COMMAND SURFACE

```
npx vitest run
npx vitest run --coverage
npx playwright test
npx playwright test --headed
```

Reported result: 7 unit test files passed, 48 unit tests passed. The E2E suite contains 11 authentication / lifecycle / billing / account-management checks.

Evaluation takeaway

The project is designed to be evaluated as a complete software-engineering artefact, not only as a working demo.

- 01 Requirements** 21 functional + 10 non-functional requirements are mapped to concrete features and constraints.
- 02 Design** Layered architecture, UML, DFD, domain model and deployment model are documented.
- 03 Implementation** Typed Server Actions, Prisma schema, Zod validation and Supabase Auth define the core structure.
- 04 Security** RBAC, middleware, server-action checks, RLS, audit logging and status restrictions provide defense in depth.
- 05 Verification** 48 unit tests + 11 E2E tests exercise business rules and route protection.



JIS

Ready for
live demo

Thank you